Queueless – Smart Queue & Appointment Management System

Name	ID	SRS Contribution
Mahamudul Hassan	23524202102	1. Introduction
Maharin Islam Momota	23524202150	2. Overall Description
Masnoon Tahim Sijan	23524202128	3. Requirements

1. Introduction

Queueless – Smart Queue & Appointment Management System is a web-based platform designed to reduce physical waiting and improve service management. It allows customers to book appointments, join virtual queues, track their queue status, while service providers can efficiently manage appointments, queues, and walk-in customers.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements of Queueless, a web-based Smart Queue and Appointment Management System.

Queueless aims to reduce unnecessary physical waiting by allowing users to book appointments, join virtual queues, receive digital tokens, monitor their queue position, and receive notifications when their turn is approaching.

The system will also provide service providers with tools to manage appointments, virtual queues, walk-in customers, service status, and basic service-performance information.

This SRS will serve as a guideline for the project development, testing, evaluation, and maintenance processes.

1.2 Intended Audience

This SRS is intended for:

- **Project Developers** – to understand and implement the required system functionality.
- **Project Manager/Team Leader** – to plan and monitor project development.
- **Quality Assurance/Testers** – to verify that the implemented system satisfies the requirements.
- **Project Supervisor/Faculty** – to review the project scope and requirements.
- **System Administrators** – to understand system management requirements.
- **End Users/Service Providers** – to understand the expected capabilities of the system.

1.3 Intended Use

Queueless is intended to be used by organizations that provide services involving appointments and/or customer queues.

Potential use cases include:

- Clinics and hospitals
- Banks and financial service centers
- Government service offices
- Salons and service centers
- Customer support centers
- Other appointment- or queue-based organizations

Customers will use the system to book appointments or join virtual queues, while service providers and staff will use it to manage customer flow and appointments.

1.4 Product Scope

Queueless will be a web-based platform consisting of three primary user roles:

1. Service Seeker/Customer
2. Service Staff/Provider
3. Administrator

The system will allow customers to:

- Create accounts and log in
- Browse available services
- Book appointments

- Join virtual queues
- Receive digital tokens
- Monitor queue position
- View estimated waiting time
- Receive queue notifications
- Cancel or reschedule appointments

Service providers will be able to:

- Manage services
- View appointments
- Manage virtual queues
- Add walk-in customers
- Call the next customer
- Skip or recall customers
- Update service status
- Monitor queue activity

Administrators will be able to:

- Manage users
- Manage service providers and staff
- Manage services
- Monitor system activity
- View basic reports and analytics

Project Goal

The primary goal of Queueless is to transform traditional physical waiting into a more efficient, transparent, and digitally managed process.

1.5 Risk Definitions

The following risks may affect the successful development or operation of the system.

Risk	Description	Mitigation
Server Failure	System may become temporarily unavailable	Use reliable hosting and regular backups

Internet Failure	Users may lose access during connectivity problems	Display clear status messages and allow service staff to handle customers manually
Incorrect Queue Data	Queue position may become inconsistent	Validate queue operations and maintain transaction records
Security Breach	Unauthorized access to user information	Use authentication, authorization, password hashing, and secure API practices
Notification Failure	Users may not receive notifications	Display queue status directly in the application as a backup
Scope Expansion	Too many features may delay development	Prioritize core MVP functionality
Concurrent Requests	Multiple users may attempt to join/update a queue simultaneously	Use proper database transactions and server-side validation

2. Overall Description

2.1 User Classes and Characteristics

2.1.1 Service Seeker - Customer

The customer is the primary user of the system.

Customers may have basic computer or smartphone knowledge and should be able to use the application without technical expertise.

Examples include:

- Patients visiting clinics or hospitals
- Customers visiting banks
- Citizens seeking government services
- Clients visiting salons or service centers
- Customers visiting customer service centers

Responsibilities:

- Register and log in
- Search for services
- Book appointments
- Join virtual queues
- Track queue status
- Receive notifications
- Cancel/reschedule appointments

2.1.2 Service Staff - Provider

Service staff are employees responsible for serving customers and managing queues.

Examples include:

- Receptionists
- Front-desk staff
- Counter officers
- Appointment coordinators

Responsibilities:

- View appointments
- Manage queues
- Add walk-in customers
- Call customers
- Skip/recall customers
- Update service status

2.1.3 Administrator

The administrator is responsible for managing the overall system.

Responsibilities:

- Manage users
- Manage service providers
- Manage services
- Monitor system activity
- View reports
- Maintain system configuration

2.2 User Needs

Customer Needs

Customers need a convenient way to:

- Avoid unnecessary physical waiting
- Book appointments easily
- Join queues remotely
- Know their queue position
- Estimate how long they need to wait
- Receive notifications before their turn
- Cancel or reschedule appointments

Service Provider Needs

Service providers need to:

- Organize appointments efficiently
- Manage walk-in customers
- Maintain a virtual queue
- Know which customer should be served next
- Update customer service status
- Monitor queue performance

Administrator Needs

Administrators need to:

- Manage users
- Control available services
- Monitor system usage
- Identify service bottlenecks
- Access basic performance reports

2.3 Operating Environment

Queueless will operate as a web-based application.

Client Environment

The system should be accessible through modern web browsers, including:

- Google Chrome

- Microsoft Edge
- Mozilla Firefox
- Safari

The application should support:

- Desktop computers
- Laptops
- Tablets
- Smartphones

Server Environment

The backend will run on a web server capable of supporting:

- Node.js
- Express.js
- REST APIs
- MySQL database

Proposed Technology Stack:

Component	Technology
Frontend	React.js, HTML5, CSS3
Backend	Node.js, Express.js

Database	MySQL
API	REST API
Version Control	Git & GitHub
UI Design	Figma
API Testing	Postman

2.4 Constraints

The following constraints apply to the project:

1. The system must be developed within the academic course duration.
2. The project will primarily focus on web-based functionality.
3. Development will be limited to the resources and technical skills available to the project team.
4. The system will require an internet connection for normal operation.
5. The initial version will focus on core queue and appointment functionality.
6. Advanced features such as AI-based waiting-time prediction and dedicated mobile applications will not be part of the initial implementation.
7. Third-party notification services may be limited depending on their availability and cost.
8. The project must remain within a manageable scope for an undergraduate course project.

2.5 Assumptions

The following assumptions are made during system development:

- Users have access to an internet-connected device.
- Users provide valid registration information.
- Service providers have predefined services and operating hours.
- Service providers will update the queue status accurately.
- The organization using the system has staff responsible for managing the queue.
- The database server is available during normal system operation.
- Users will check the application or enabled notifications to monitor their queue.
- The initial system will be deployed for a limited number of organizations/services during testing.

3. Requirements

The requirements of a project describes how viable ,efficient and accurate format the project is.

3.1 **Functional Requirements:** The functional requirements describe what the system must do.

I. **User Registration:**

I want to create an account so that I can access the virtual appointment and queue service.

Confirmation

- The system shall provide a registration form.
- The customer shall provide required information such as name, email/phone number, and password.
- The system shall validate the provided information.
- The system shall prevent duplicate accounts using the same registered identifier.
- A successfully registered user shall be able to log in.

II. **User Login**

As a registered user, I want to log into the system so that I can access my account and services.

Confirmation

- The system shall provide a login interface.
- The user shall provide valid login credentials.
- The system shall authenticate the credentials.
- The system shall provide access according to the user's role.
- Invalid credentials shall generate an appropriate error message.

III. Appointment Booking

As a customer, I want to book an appointment so that I can receive service at a selected time.

Confirmation

- The system shall display available appointment slots.
- The customer shall select a date and time.
- The system shall verify slot availability.
- The system shall create the appointment after successful confirmation.
- The customer shall receive an appointment confirmation.
- The appointment shall appear in the customer's appointment history.

IV. Virtual Queue Joining

As a customer, I want to join a virtual queue so that I do not have to physically wait at the service location.

Confirmation

- The system shall allow eligible customers to join an active queue.
- The system shall generate a unique queue/token number.
- The system shall record the customer's queue entry time.
- The system shall display the customer's current queue position.
- The system shall prevent duplicate active queue entries for the same service.

V. Digital Token Generation

As a customer, I want to receive a digital token so that I can identify my position in the queue.

Confirmation

- The system shall generate a unique token for each queue entry.
- The token shall be associated with the customer and selected service.
- The token shall be displayed in the customer's dashboard.
- The token status shall be updated as the queue progresses.

VI. Real-Time Queue Tracking

As a customer, I want to receive a notification when my turn is approaching so that I can prepare to receive the service.

Confirmation

- The system shall monitor the customer's queue position.
- The system shall generate an in-system notification when the customer's turn is approaching.
- The notification shall identify the relevant queue/token.
- The system shall update the notification status after the customer is called.

VII. Appointment Cancellation and Rescheduling

As a customer, I want to cancel or reschedule my appointment so that I can manage unexpected changes in my schedule.

Confirmation

- The system shall allow cancellation of eligible appointments.
- The system shall allow rescheduling where slots are available.
- Cancelled appointments shall no longer occupy the original slot.
- The system shall update the appointment status.

VIII. Staff Dashboard

As a service staff member, I want to access a dashboard so that I can manage appointments and queues efficiently.

Confirmation

- The dashboard shall display active queues.
- The dashboard shall display upcoming appointments.
- The staff shall be able to view customer tokens.
- The staff shall be able to update queue status.
- Dashboard information shall be restricted to authorized staff.

3.2 Non-Functional Requirments

I. Performance

- The system should respond to normal user requests within an acceptable time.
- Common page requests should ideally load within 3 seconds under normal conditions.
- Queue updates should be reflected with minimal delay.
- The system should support multiple users accessing the application simultaneously during normal expected usage.

II. Security

- Users shall be authenticated before accessing protected features.
- Passwords shall not be stored as plain text.
- Role-based authorization shall be implemented.
- Customers shall not be able to access staff or administrator functions.
- Sensitive information shall be protected from unauthorized access.
- API endpoints shall validate incoming requests.

III. Reliability

- The system shall maintain consistent queue information.
- The system shall prevent duplicate active queue entries.
- Appointment and queue updates shall be stored correctly.
- The system should recover gracefully from common application errors.

IV. Maintainability

- The application should use modular and organized code.
- Frontend, backend, and database components should be logically separated.
- Meaningful naming conventions should be followed.
- Source code should be maintained using Git/GitHub.
- Documentation should be maintained for major system components.